

Machine Learning

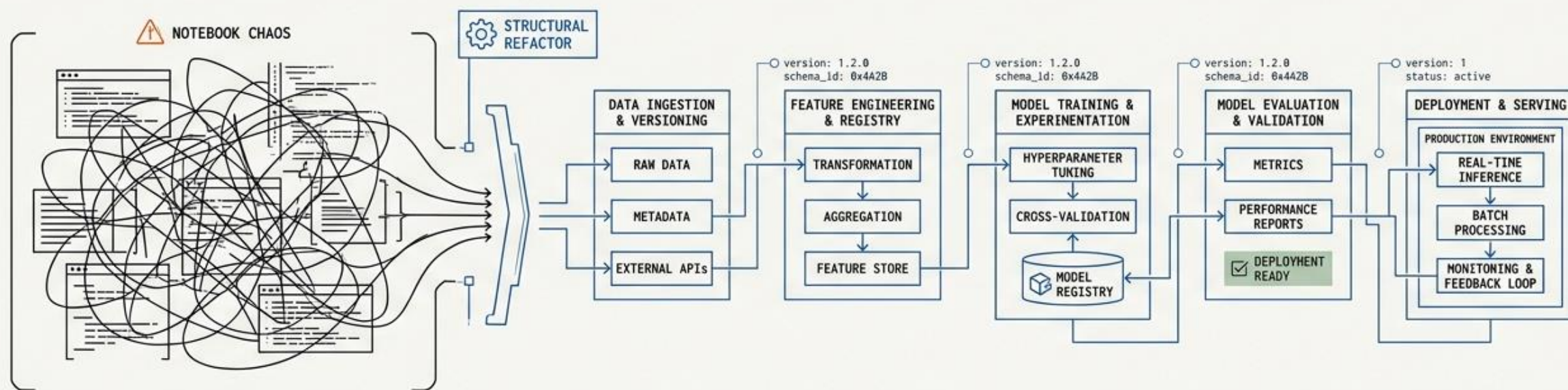
Week 12 Lecture

Dr. Muhammad Ahmed Raza

Contents

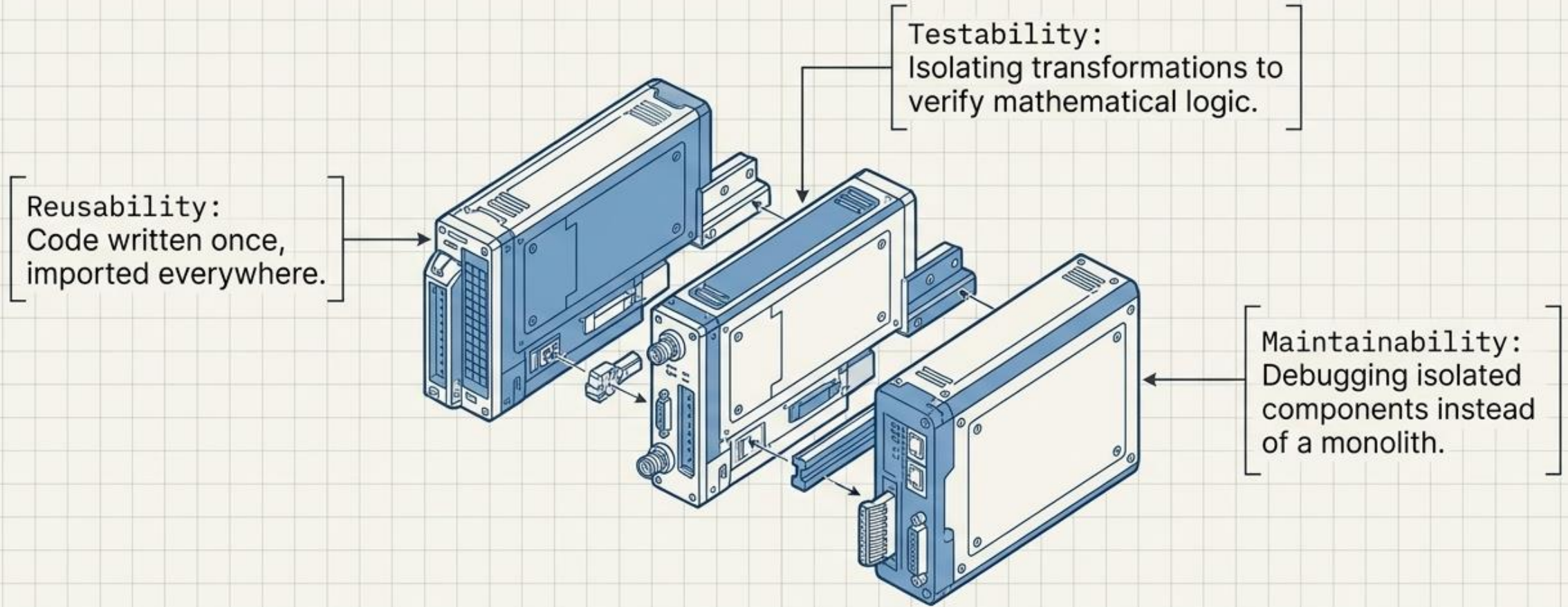
- MLFlow
- Introduction
- MLOps fundamentals
- Model Serialization
- Rest APIs
- Fast API

Architecting the Production ML Pipeline



> Executing structural refactor...
moving from notebook chaos to tracked,
reproducible systems. ■

Pillar 1: Modular Software Design



[Every script must do exactly one thing well.]

Enforcing the Separation of Concerns



The Anti-Pattern

`model_final_v3_really_final.ipynb`



The Professional Tree



`data.py`



`features.py`



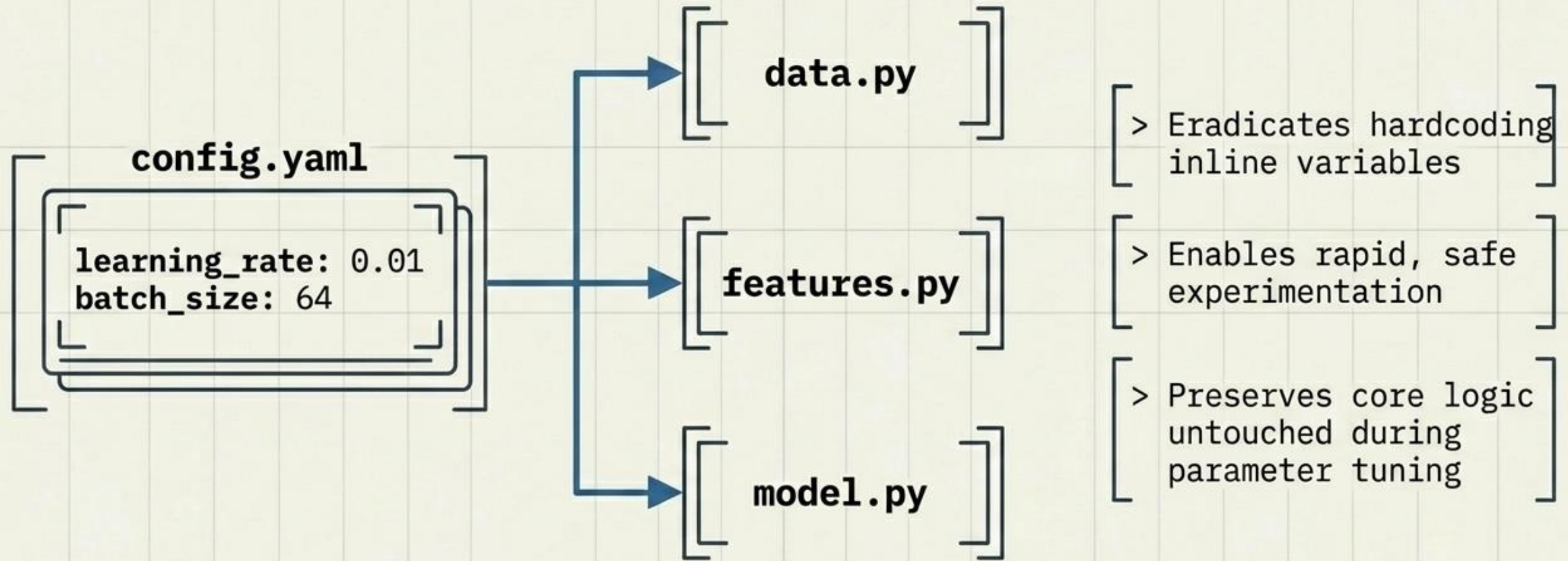
`model.py`



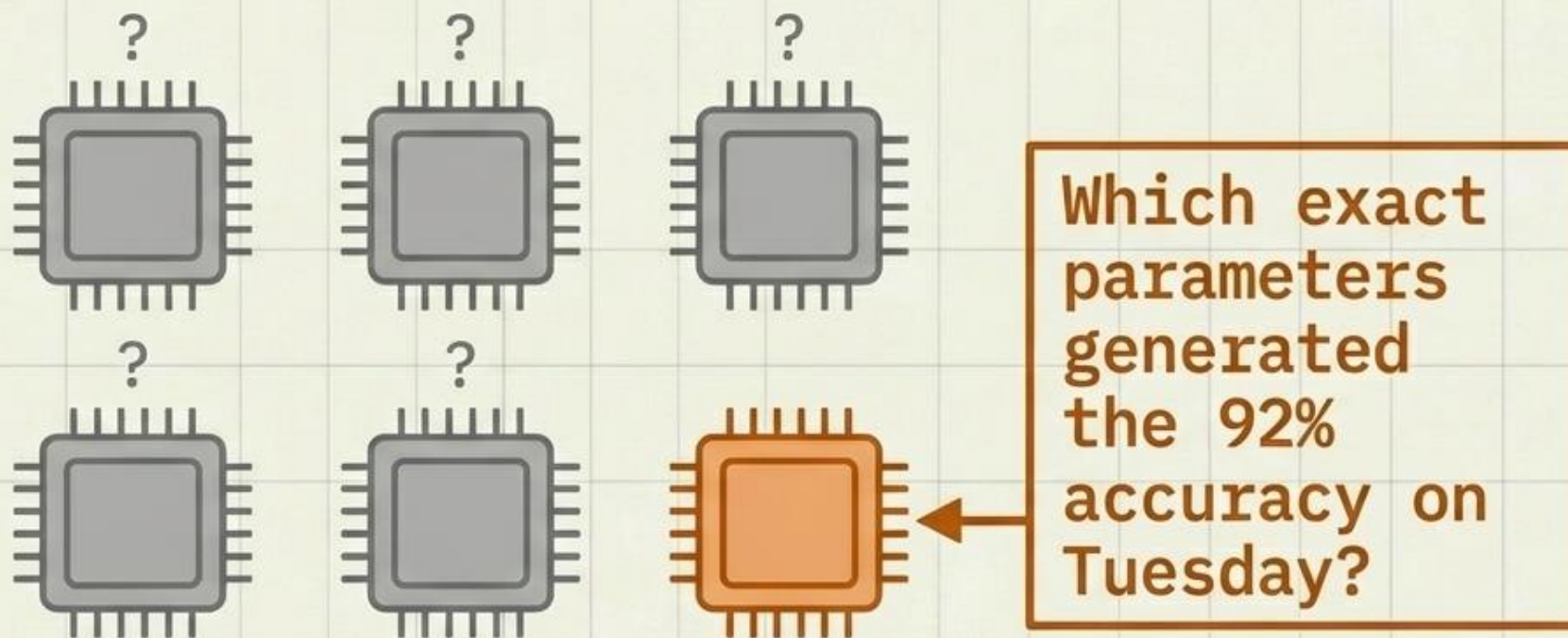
`train.py`

Orchestration
is strictly
decoupled from
mathematical
logic.

The Extensibility of Config-Driven Architecture

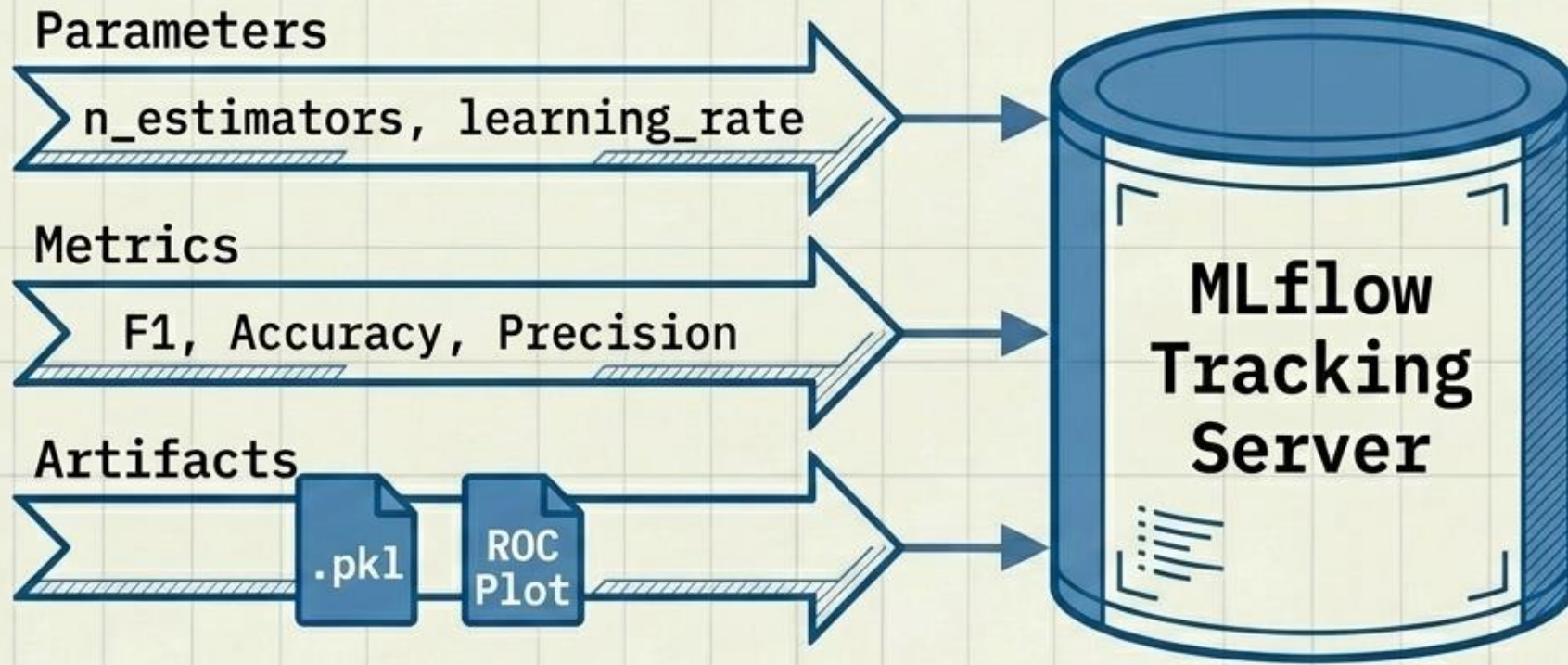


Pillar 2: The Experimentation Graveyard



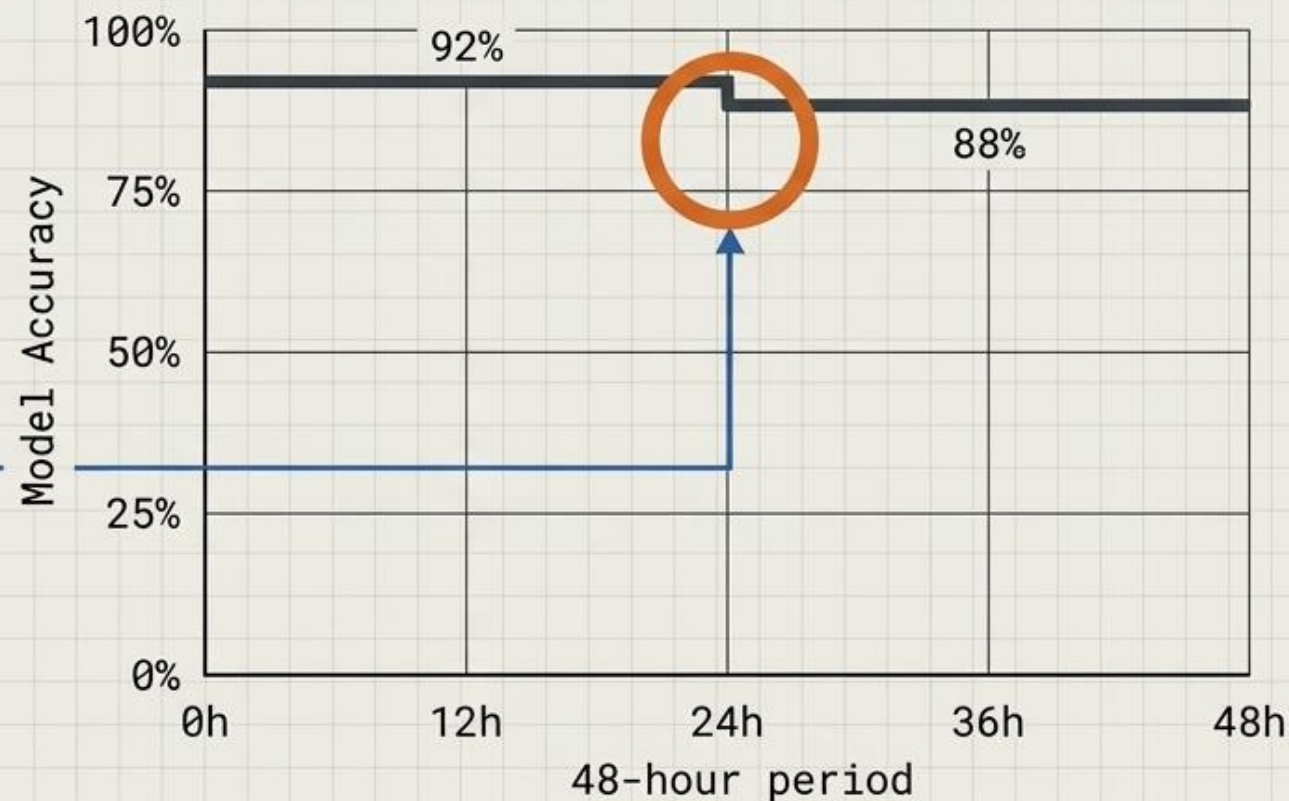
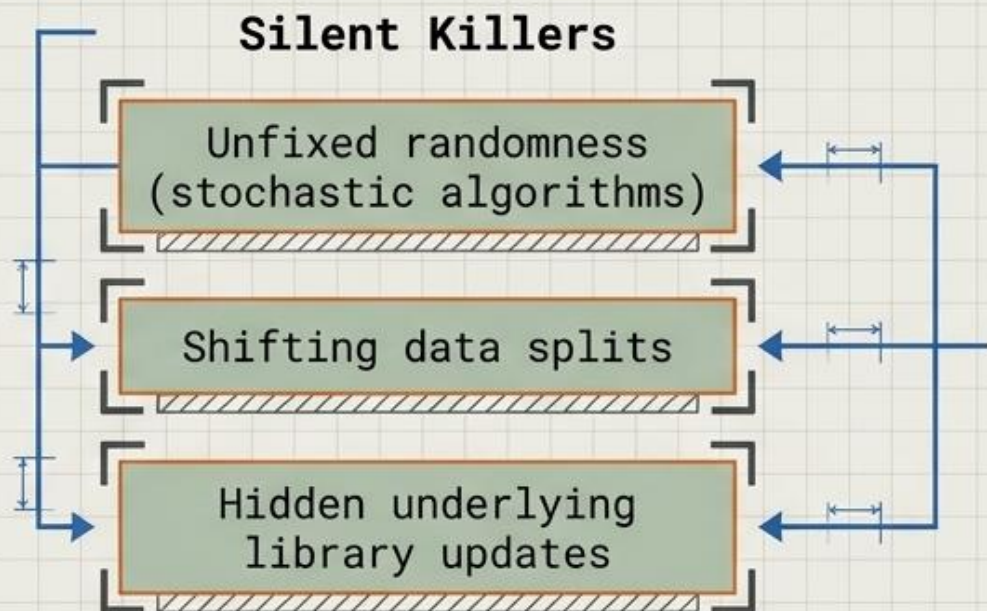
Manual tracking fails. Without an automated ledger, experimental history is permanently lost the moment a cell is re-run.

Automated Memory via MLflow



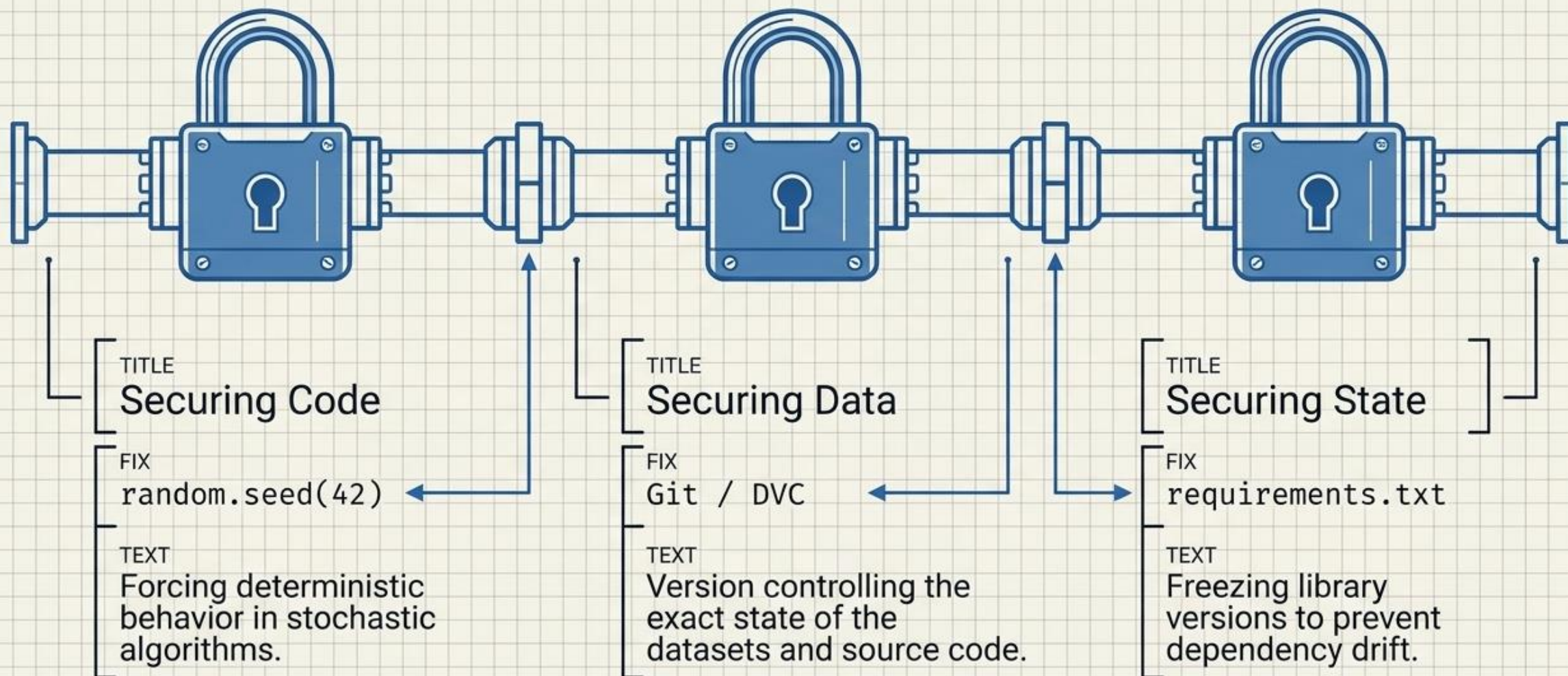
MLflow acts as the immutable system of record, replacing manual notes with queryable, automated tracking.

Pillar 3: The Reproducibility Crisis



Reproducibility: The baseline requirement to generate the exact same mathematical results reliably across time and space.

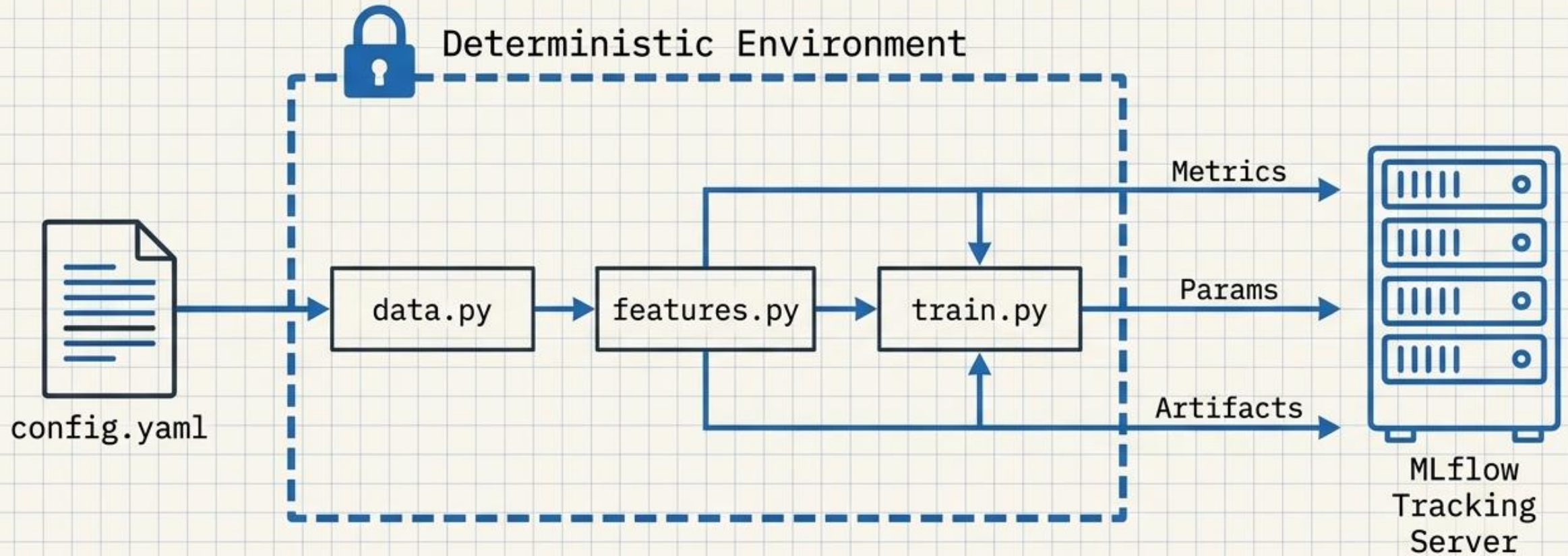
Locking Down the Pipeline Environment



Diagnostic Matrix: Assessing Pipeline Maturity

The Interactive Script	The Production Pipeline
Hardcoded inline variables	External YAML configs
Relies on human memory	Automated MLflow ledger
Manual cell-by-cell execution	Single-command orchestration
Stochastic & fragile	Deterministic & locked

Synthesis: The Modern ML Workflow Blueprint



A deterministic environment executing modular code, driven by centralized configuration, outputting continuous telemetry. CL0-3 achieved.

Case Study: The Responsible Competition

MLflow run-comparison			
Model	Dataset Version	Seed	F1 Score
Logistic Regression	v2.1	42	0.81
Random Forest	v2.1	42	0.87
XGBoost	v2.1	42	0.91

Variables locked.

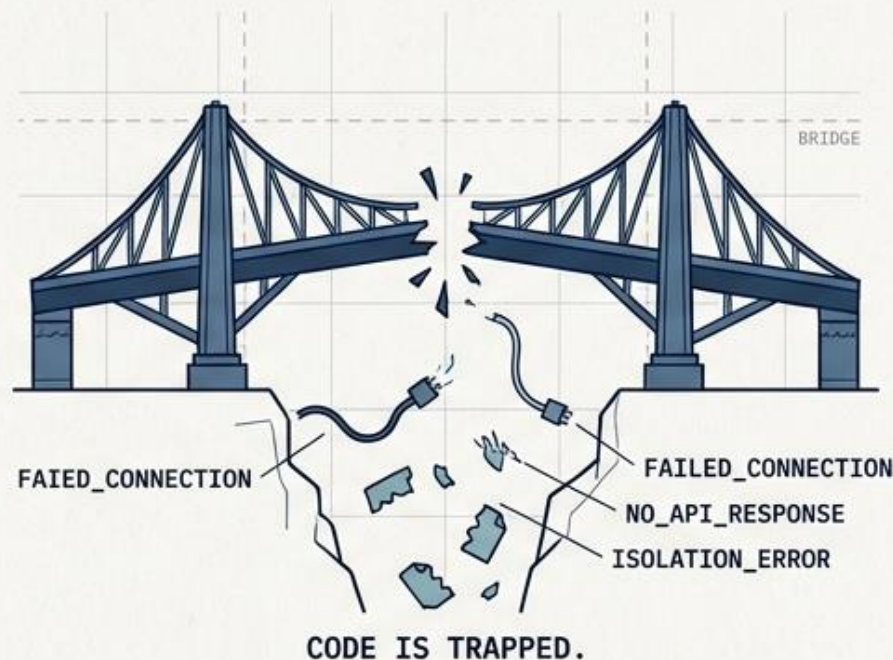
Clean evaluation.

This architecture makes evaluating distinct inductive biases effortless, transparent, and completely defensible.

Architecture Secured. Awaiting Deployment.

[OK] Pipelines > Scripts
[OK] Modularity = Scale
[OK] MLflow = System Memory
[OK] Reproducibility = Credibility

"A highly accurate model is useless if applications cannot access it."



The Local Trap



Training is only the development phase. Code living in local `.ipynb` files cannot communicate with web apps, mobile devices, or other software systems.



The Deployment Imperative



MODEL_PACKAGE

Packaging and serving transform a mathematical algorithm into a network-accessible service.



The Deployment Gap — Why It Matters

The Problem

- 87% of ML models never reach production (Gartner)
- Models trained in notebooks can't run in the real world
- No deployment = no business value
- Teams waste months on models no one can use

The Goal

- Model accessible via API — callable from any app
- Scales to handle thousands of requests per second
- Versioned, monitored, and updatable without downtime
- Reproducible across dev, staging, and production

Your model is only useful if others can use it!

Real-World ML Deployment Examples



Netflix — Recommendation Engine

Trained models recommend shows; deployed APIs serve predictions per user in real-time.



Banking — Fraud Detection

Transaction data hits an API → ML model scores risk → approve or flag in milliseconds.



Healthcare — Diagnostic AI

X-ray image uploaded → served model outputs diagnosis → doctor reviews alert.



E-commerce — Dynamic Pricing

Demand model updates prices hourly; deployed via scheduled inference jobs.



Mobile Apps — On-device ML

TensorFlow Lite / CoreML — models packaged and served inside the app itself.



Chatbots — NLP Models

LLMs served behind REST APIs; front-end calls /chat, gets response, renders it.

What is MLOps?

MLOps = ML + DevOps + Data Engineering

A set of practices that aims to deploy and maintain ML models in production reliably and efficiently.



ML (Science)

Training, tuning, and evaluating machine learning models



DevOps

CI/CD, automation, containers, monitoring in production

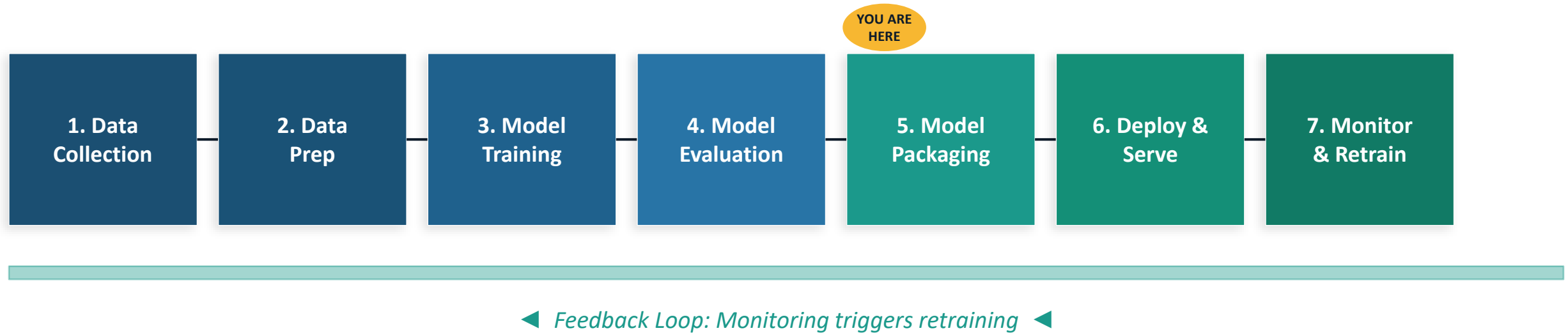


Data Eng.

Pipelines, versioning, data quality, feature stores

💡 *Analogy: MLOps is to ML what DevOps is to software engineering — it turns experiments into products.*

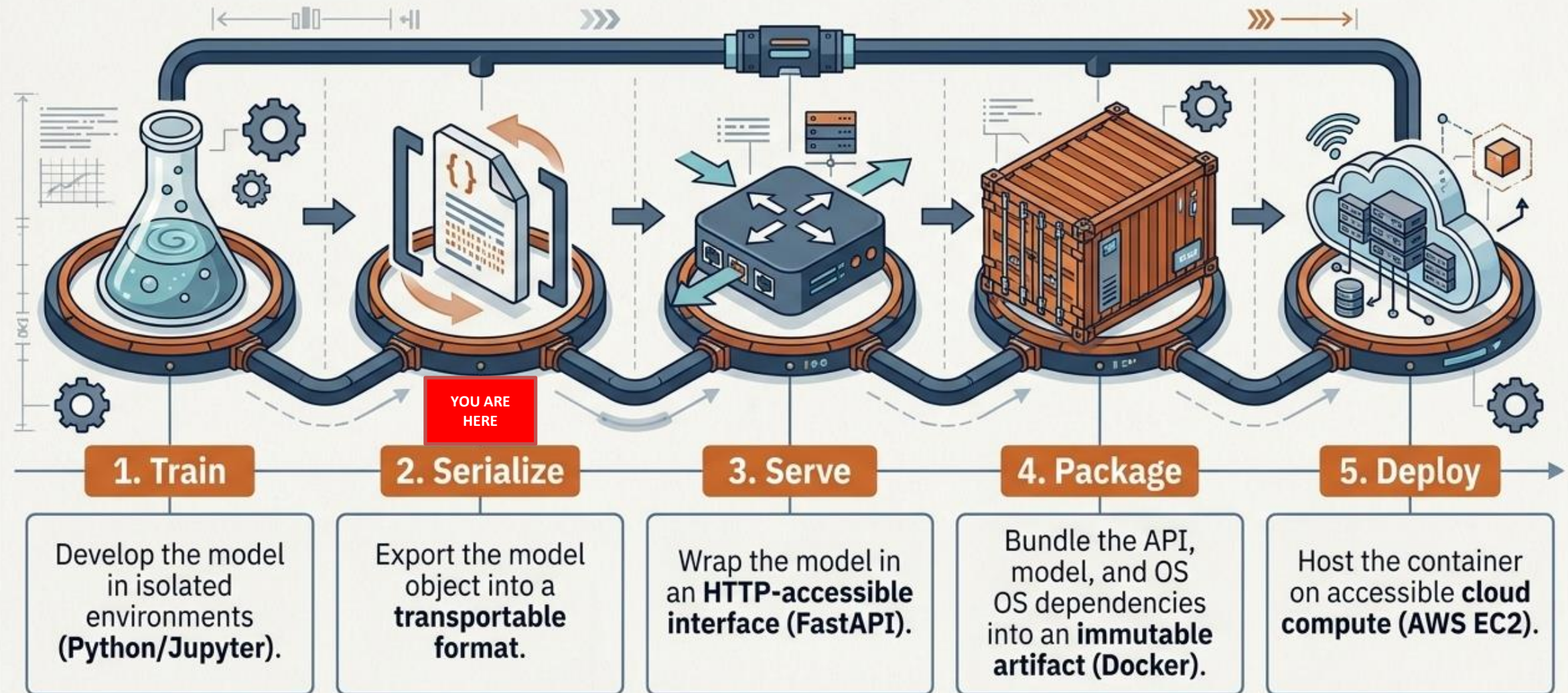
The MLOps Lifecycle — End-to-End Pipeline



This lecture focuses on stages 5–6: Packaging & Deployment

Business Problem → Data → Model → **API Endpoint** → **Application** → User

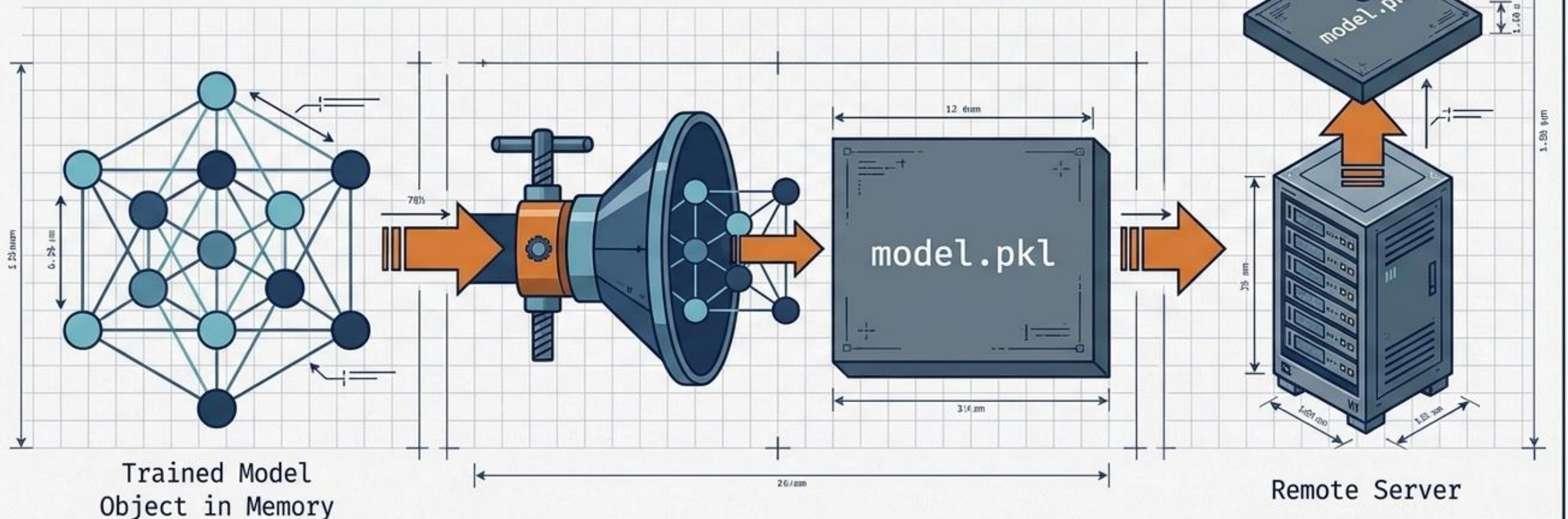
The Deployment Logistics Pipeline



Serialization: The Freeze and Thaw

Why Serialize?

Prevents having to retrain models from scratch upon deployment. It captures the learned weights, biases, and parameters, saving them directly to disk.



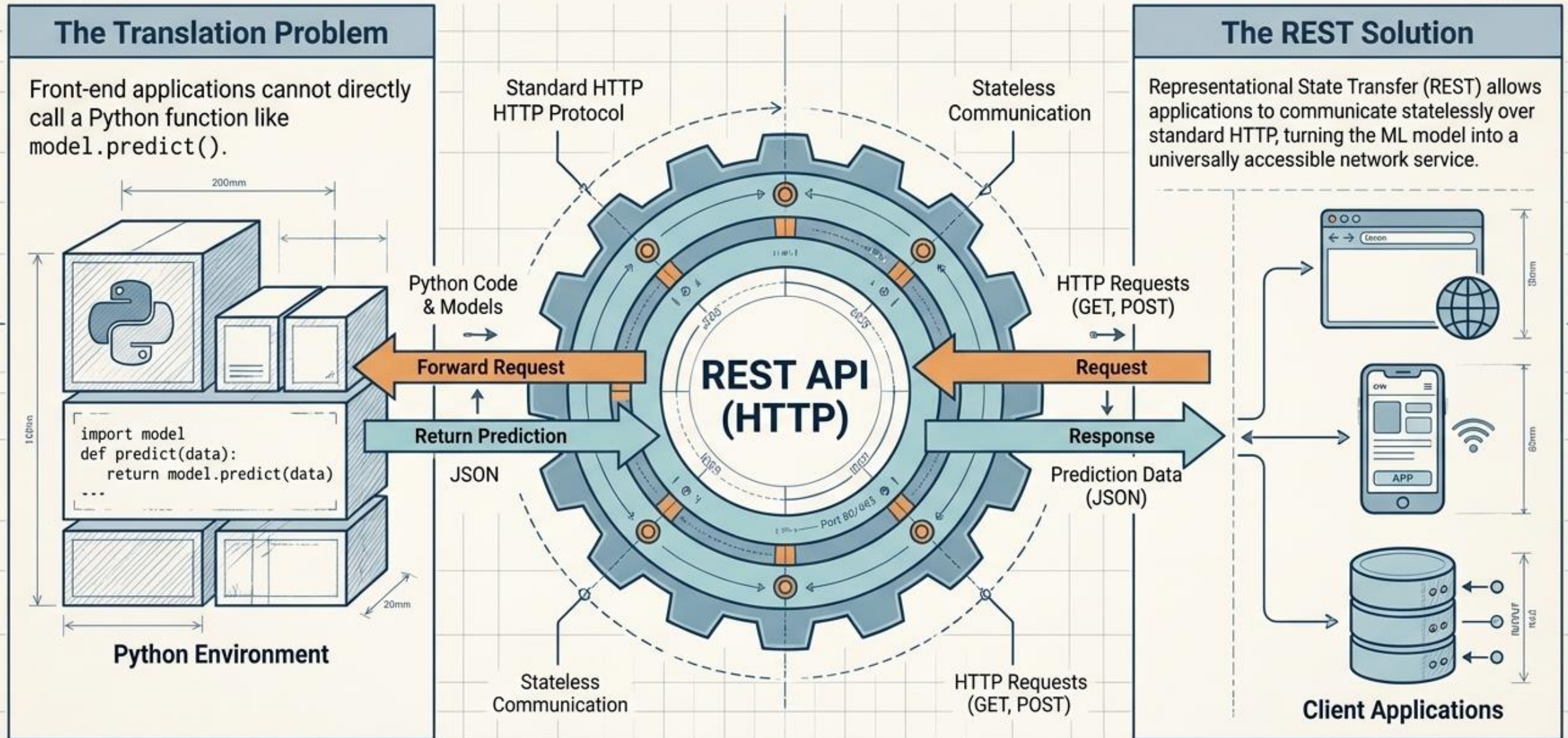
The Serialization Showdown

	Pickle (Standard Library)	Joblib (Third-Party)
Primary Use Case	General Python object serialization	Highly efficient for large NumPy arrays (vital for ML models)
Performance	 Average	 High
Output Format	.pkl	.joblib



Security Warning: Both Pickle and Joblib can execute arbitrary code during unpickling. Never load serialized models from untrusted sources. Version matching (**Python & Library**) between training and deployment is strictly required.

The API Bridge: Translating Python to the Web



The Anatomy of an HTTP Dialogue

The Methods

GET

Retrieve data (Read)

POST

Send data for processing
(Create/Predict)

PUT

Update existing data

DELETE

Remove data

The Responses

200 OK

Request succeeded.

400 Bad Request

Invalid input data.

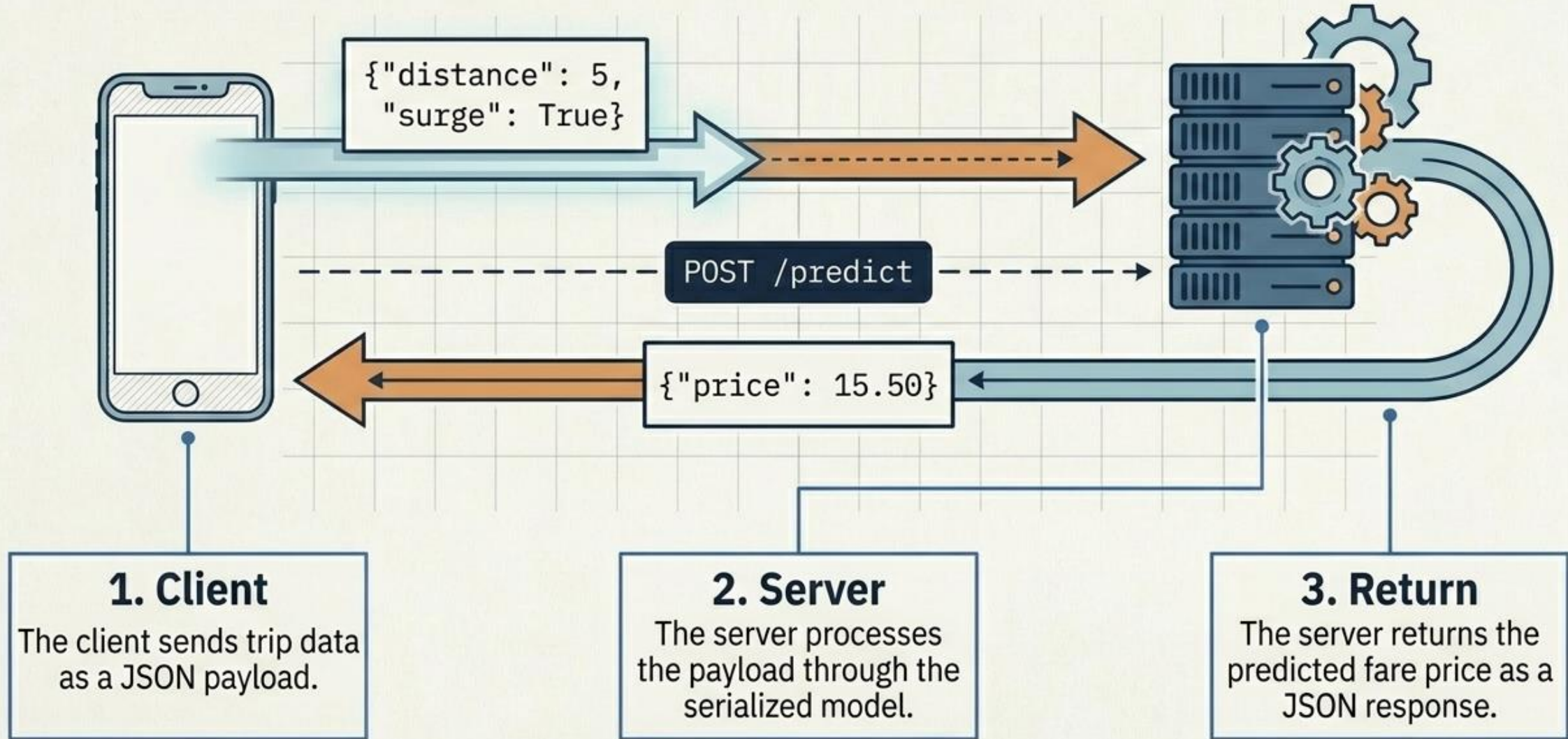
404 Not Found

Endpoint does not exist.

500 Server Error

Pipeline failure /
Exception.

Request and Response in Action



FastAPI — Building a /predict Endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI()
model = joblib.load("model.pkl")

class Features(BaseModel):
    data: list[float]

@app.post("/predict")
def predict(features: Features):
    pred = model.predict([features.data])
    return {"prediction": pred[0]}
```

 Import FastAPI
& Pydantic model

 Load serialized
ML model

 Define input
schema (validated)

 POST endpoint
at /predict

 Return JSON
prediction

Running, Testing & Swagger Docs

1

Install & Run

```
pip install fastapi uvicorn
uvicorn main:app --reload
```

uvicorn is the ASGI server that runs FastAPI. `--reload` auto-restarts on code changes.

2

Test with curl

```
curl -X POST "http://127.0.0.1:8000/predict" -H
"Content-Type: application/json" -d
"{\"features\": [5.1, 3.5, 1.4, 0.2]}"
```

Sends a POST request with JSON body. Returns your model's prediction.

3

Swagger UI

```
http://127.0.0.1:8000/docs
```

FastAPI auto-generates an interactive API explorer — test endpoints directly in the browser!



Swagger UI is at `/docs` and ReDoc at `/redoc` — both auto-generated with zero extra code!



Quick Check!

Interactive Question

A user uploads an image to your app.
The app sends it to your ML model and gets a result.
What HTTP method should the app use?

A) GET

B) POST

C) DELETE

D) PUT

POST — because we're sending data (the image) to the server to process it.

Difference between APIs

- REST API → a concept / architecture style
- FastAPI → a tool/framework to build APIs
- Think of it like this:
 - REST API = **rules of the road**
 - FastAPI = **the car**

How They Work Together

- You are:
 - Using FastAPI
 - To build a REST API endpoint

Aspect	REST API	FastAPI
Type	Architecture style	Framework
Purpose	Defines communication rules	Implements APIs
Language	Any	Python
Example	/predict endpoint	Code to create /predict